AI Lakehouse Design report

- What did I build ?

I set up a versioned “data lakehouse” using three tools that work together. DuckLake serves as a catalog, which lists all my tables and their history. RustFS is the object storage ; an S3-style “bucket” where the actual files reside. DuckDB is the engine that executes SQL queries.

The data passes through three layers. “Raw” refers to the data as it was imported. “Silver” refers to the cleaned and typed data. “Gold” refers to the final table ready for machine learning. I followed this process for two datasets: COCO, which contains images, and VisDrone, which contains videos captured by drones.

- Q1: Why split the catalog from the data files ?

The catalog is a tiny file that only stores info about my tables, like what columns exist and which files belong to which version. The real data sits separately as Parquet files in RustFS. The point is that they scale on their own. The engine reads the small catalog to figure out a query plan, then only grabs the data files it actually needs. I attach it with ATTACH 'ducklake:metadata.ducklake' AS lake (DATA_PATH 's3://lakehouse/'), so the catalog is local and the data is remote.

- Q2 : How does versioning work ?

Every change makes a new snapshot, and nothing ever gets overwritten. When I added more COCO data, the table went from 219 rows at snapshot 16 to 273 rows at snapshot 17, and the old version stayed exactly as it was. That means time travel just works. I can run SELECT ... AT (VERSION => 16) and get the old 219 row table back. Rollback is the same idea. I just point the table at an older snapshot, since all the old files still exist. FROM ducklake_snapshots('lake') shows the full history.

- **Q3 : How do you keep data clean without primary keys ?**

DuckLake does not enforce keys or constraints, so I handle quality in the SQL itself, one layer at a time. In silver I remove duplicates with SELECT DISTINCT, fix the types with CAST, and drop bad rows with WHERE bbox IS NOT NULL. In gold I roll it up to one clean row per image and add a train and val split. If I ever find a bug, I just fix the SQL and rerun, and the bad version stays in history as a record**.**

- **Q4 : What actually happens on an INSERT ?**

When I ran the command `INSERT INTO raw.coco_annotations`, three things happened. A brand-new Parquet file was written to RustFS with the new rows. The catalog recorded a new snapshot pointing to that file. And all the old files remained intact.

Thus, an insertion never modifies existing data. It simply adds new files and increments the version pointer. This is exactly why “time travel” is practically free.

- **Q5: Why use a SQL catalog instead of one big file?**

If a table were just a giant file, or a simple folder containing Parquet files, there would be no easy way to know what it looked like last week, or which files were modified in snapshot #17.

An SQL catalog stores all this history in the form of rows that I can query. It also allows me to benefit from clean atomic commits, where a snapshot is either fully committed or not at all, as well as secure reads during writes, and a complete schema history, all using standard SQL.

- **Q6 : Why keep images and video out of the tables?**

Images and videos are large binary blocks. Embedding them directly into table rows would make each file enormous and slow down every query. Instead, the actual bytes are stored in an object storage system, for example s3://lakehouse/assets/coco/139.jpg, and the table stores only the link along with metadata such as tags, selection frames, and sizes.

For VisDrone, I took it a step further. Video is stored as small fragments, and an index table contains statistics for each fragment. As a result, my query for the most frequently accessed fragments reads only 6 fragments about 12 KB instead of 24. It retrieves only the bytes it
needs.

- **Results :** https://huggingface.co/datasets/jeangardy810/lakehouse-coco-gold